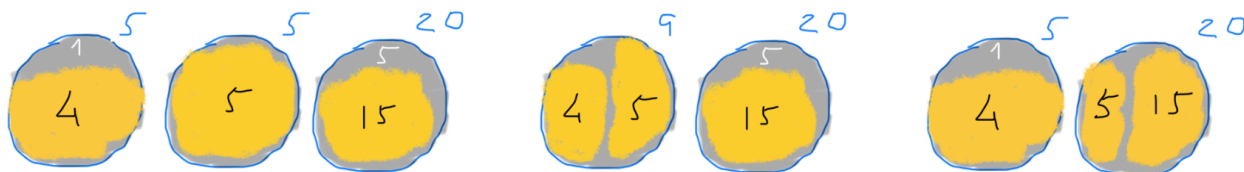


Relatório sobre o 1º trabalho prático de EDA 2

The Dream Factory



Universidade: Colégio Luís António Verney (Universidade de Évora)

Curso: Engenharia Informática

Identificações: Grupo g115

(Diogo Tavares N:58049 ,Francisco Rodrigues N:59119)

Ano: 2024/25

Descrição do Algoritmo

- **linhas 7 -27 Leitura e processamento de entradas:**

Começamos o código pela leitura do input fornecido e respetiva associação às variáveis indicadas no enunciado.

Lemos o número de numeros disponiveis para guardar os sonhos que serão encomendados(N) e depois criamos e ordenamos(`Arrays.sort`) um array de inteiros($ni[]$) de tamanho previamente lido(N) para guardar estes número, após isso criamos também uma variável ($maxNi$) com o maior número disponível para guardar sonhos.

De seguida lemos o número de sonhos encomendados(D) e depois criamos outro array de inteiros(dj), de tamanho D , com os respectivos números dos sonhos encomendados.

Após termos as variáveis prontas passamos a carregar as designadas variáveis com o seu valor respectivo do input.

- **Linhas 34-61 Função M:**

A função iterativa M utiliza programação dinâmica (abordagem bottom-up) para calcular o desperdício mínimo ao agrupar os sonhos encomendados (dj) nos números disponíveis (ni).

Construindo uma tabela de estados (dp), onde cada posição $dp[i]$ armazena o desperdício mínimo acumulado ao processar os primeiros i sonhos.

A solução é otimizada através de buscas binárias no array ordenado ni , garantindo eficiência na seleção do número ideal para cada grupo de sonhos.

- **Inicialização do Array de dp (Desperdício):**

$long[] dp$ que seria o desperdício mínimo para cada quantidade de sonhos, inicialmente todos os valores são definidos como

$long.MAX_VALUE$ (que seria um valor infinito).

$dp[0]$ é definido como 0 porque é o caso base que seria quando há sonhos encomendados ou seja não há desperdício

- **Iteração sobre os sonhos:**

Para cada quantidade de sonhos de 1 até D (tamanho de dj), a função calcula o desperdício mínimo.

sum acumula a soma dos tamanhos dos sonhos de j até $i-1$.

Se sum exceder $maxNi$, o loop interno é interrompido, pois não é possível acomodar mais sonhos sem exceder o maior número disponível.

- **Busca e cálculo do dp(Desperdício):**

A função usa `Arrays.binarySearch` para encontrar a posição do menor número em `ni` que pode acomodar a soma `sum`.

Se a posição retornada for negativa, ajusta-se para um índice válido.

Se a posição for maior ou igual ao tamanho de `ni`, ignora-se essa soma.

Calcula-se o desperdício como a diferença entre `ni[pos]` e `sum` e depois atualiza-se `dp[i]` com o menor desperdício encontrado.

Depois a função retorna o valor em `dp[dj.length]`, que representa o desperdício mínimo para acomodar todos os sonhos.

- **Conclusão da Função M:**

A função `M` utiliza programação dinâmica para calcular o desperdício mínimo ao acomodar uma série de sonhos em números disponíveis, considerando todas as combinações possíveis e otimizando o desperdício em cada etapa.

Exemplo prático da função M a calcular o desperdício

iteração (i)	j	Sonhos Processados (dj[j...i-1])	Soma Atual (sum)	Busca Binária (pos)	Desperdício	Atualização do dp[i]	Estado do dp
Inicialização	-	-	-	-	-	$dp = [0, \infty, \infty, \infty]$	$[0, \infty, \infty, \infty]$
i = 1	0	[4]	4	0 (5)	1	$dp[1] = \min(\infty, 0 + 1) = 1$	$[0, 1, \infty, \infty]$
i = 2	1	[5]	5	0 (5)	0	$dp[2] = \min(\infty, 1 + 0) = 1$	$[0, 1, 1, \infty]$
i = 2	0	[4, 5]	9	1 (9)	0	$dp[2] = \min(1, 0 + 0) = 0$	$[0, 1, 0, \infty]$
i = 3	2	[15]	15	2 (20)	5	$dp[3] = \min(\infty, 0 + 5) = 5$	$[0, 1, 0, 5]$
i = 3	1	[5, 15]	20	2 (20)	0	$dp[3] = \min(5, 1 + 0) = 1$	$[0, 1, 0, 1]$
i = 3	0	[4, 5, 15]	24 (>20)	-	-	Loop interrompido	$[0, 1, 0, 1]$ (resultado)

Complexidade Espacial

- **Complexidade durante a leitura:**

Durante a leitura e processamento do input as únicas variáveis significativas para a complexidade espacial são os arrays ni e dj . O array ni ocupa $O(N)$ e o dj ocupa $O(D)$. Logo a seguir a ordenação do array ni ocupa $O(N)$.

- **Complexidade da Função M:**

O array dp na função M tem tamanho $D + 1$, ocupando $O(D)$, logo a função M só tem complexidade espacial de $O(D)$.
Logo a Complexidade espacial total é de $O(N + D)$.

- **Complexidade espacial total:** Assim obtemos

$$O(N) + O(D) + O(N) + O(N + D) = \\ O(N + D)$$

Complexidade Temporal

- **Complexidade durante a leitura:**

na função main na linha 19-21 os for loops para leitura do input têm complexidade temporal de $O(N)$ e os das linhas 25-27 têm complexidade $O(D+N)$, pois um for loop está a ler e a processar D elementos e o outro N elementos.

- **Complexidade do `Arrays.sort(ni)`:**

Na linha 22 é utilizado o `Arrays.sort(ni)`. Como em java o sort utiliza um algoritmo de ordenação de complexidade temporal $O(N \log N)$ onde N é o tamanho do array a ser ordenado, e o tamanho do array `ni` é a variável N , então a complexidade temporal da função `Arrays.sort(ni)` é **$O(N \log N)$** .

- **Complexidade da Função M:**

A função M tem dois for loops aninhados que têm uma complexidade de $O(D)$ cada um, logo esses 2 for loops têm uma complexidade $O(D^2)$, sendo `dj.length = D`.

Depois ainda há o `BinarySearch` na linha 50 tem a complexidade de $\log(n)$.



- **Complexidade Temporal total:**

Juntando todas as complexidades previamente explicadas dentro da função M obtemos:

$$\begin{aligned} &O(D+N) + O(N \log N) + O(D^2) + O(D^2 \log N) \\ &= \\ &O(N \log N + D^2 \log N) \end{aligned}$$